Assignment 4 - Dynamic Programming and String Algorithms

Zhangirkhan Temirlan SE 2521

GitHub: GitHub - Legwnsa/Assigment-4

1) Experiment Design Card

Card field	Content
Question	**Edit Distance:** does the recursive version really grow exponentially while the table and two-row versions grow like n·m? How much memory do two rows save, and does it change the running time? **String search:** does Naive degrade to O(n·m) on a bad input while KMP stays O(n+m)? Does KMP also help on ordinary random text?
Sizes (n)	**Edit Distance, all three versions:** n = m ∈ {3, 6, 9, 12} – the recursion cannot handle much more, and 12 is 4× the smallest. **Table vs two rows:** n = m ∈ {500, 1000, 2000, 4000} – up to 16 million cells, tens of milliseconds, so the clock is not 0. **String search:** text length n ∈ {100 000, 200 000, 400 000, 800 000, 1 600 000}, pattern length fixed at m = 100, so only n changes.
Inputs	**Edit Distance:** `random` – random strings over {A,C,G,T}, the typical case; `identical` – a = b, best case for the recursion (only one path, nothing to branch on); `disjoint` – all `A` against all `B`, no equal characters ever, so the recursion branches three ways in every call (worst case). **String search:** `random` – random text and random pattern over {A,C,G,T}, the usual case; `worst_case` – text `A…A`, pattern `A…AB`, so Naive compares almost the whole pattern at every start position; `all_match` – text `A…A`, pattern `A…A`, every position matches (overlapping matches). All strings come from my own generator with a fixed seed.
Metrics	**time_ms** – median of 5 runs after warm-up (3 runs for the recursion because it is slow), shows what a user would feel. **ops** – Edit Distance: subproblems evaluated (recursive calls / table cells), because this is the work that DP removes; string search: character comparisons (including the LPS build), because this is the work that KMP removes. **memory_cells** – array cells allocated (recursion: maximum recursion depth), to show the price of the table. **result** – the distance or the number of matches, as a sanity check.
Prediction	**Edit Distance:** recursive calls on `disjoint` multiply by about 5.8 for every +1 in n (theory: the number of paths grows like (3+2√2)^n), so about ×200 for +3; on `identical` they are n+1 (a straight line); `random` lies between. Table and two rows do exactly (n+1)·(m+1) cell evaluations on every input, so `ops` is equal for both and independent of the input type; time grows ×4 when n doubles; two rows are as fast as the table or slightly faster. Memory: (n+1)² cells for the table against 2(n+1) for two rows, a ratio of about n/2 (≈2000× at n = 4000). **String search:** on `random` Naive needs about 1.33·n comparisons and KMP about n to 1.3·n, so they are almost equal and the times are close. On `worst_case` and `all_match` Naive needs about m·n = 100·n comparisons and KMP about 2·n, a ratio near 50; Naive time is clearly larger. For both, time doubles when n doubles.
Plots	P1 `ED_ops.png` – does the recursion grow exponentially while table and two rows do not? (subproblems vs n, log axis). P2 `ED_time.png` – how do table and two rows grow in time? (time vs n, log-log). P3 `ED_memory.png` – how much memory do two rows save? (memory cells vs n, log axis). P4 `Search_ops.png` – on which input does Naive lose to KMP in comparisons? (comparisons vs n, log-log). P5 `Search_time.png` – does the comparison count show up in the running time? (time vs n, log-log).

Changes to the card aftere the first run: none. All counters are deterministic (same seed), so they are the same on every machine; the times in this report come from the run saved in results/results.csv

2. **Plots and Results**



Figure 1. Look at the log axis: the recursion on disjoint strings grows by about ×5.6 per extra character (377 million calls at n = 12), while table and two rows stay at $(n+1)^2 = 169$ cells.

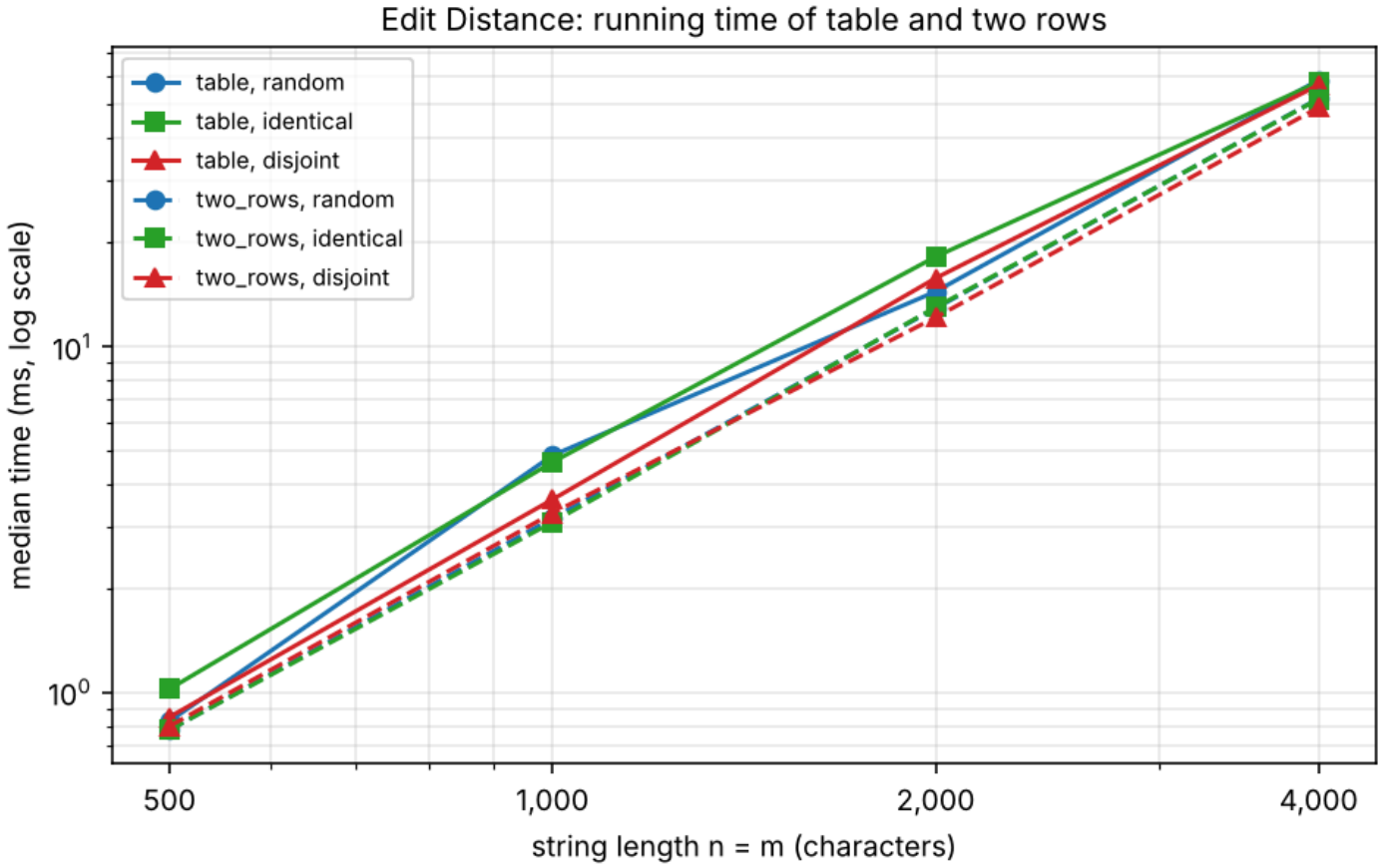


Figure 2. Two rows are faster than the table in all 12 pairs of runs (at n = 4000: 49–52 ms against 57–59 ms), and the time of two rows grows ×4.0– 4.1 for every doubling of n.

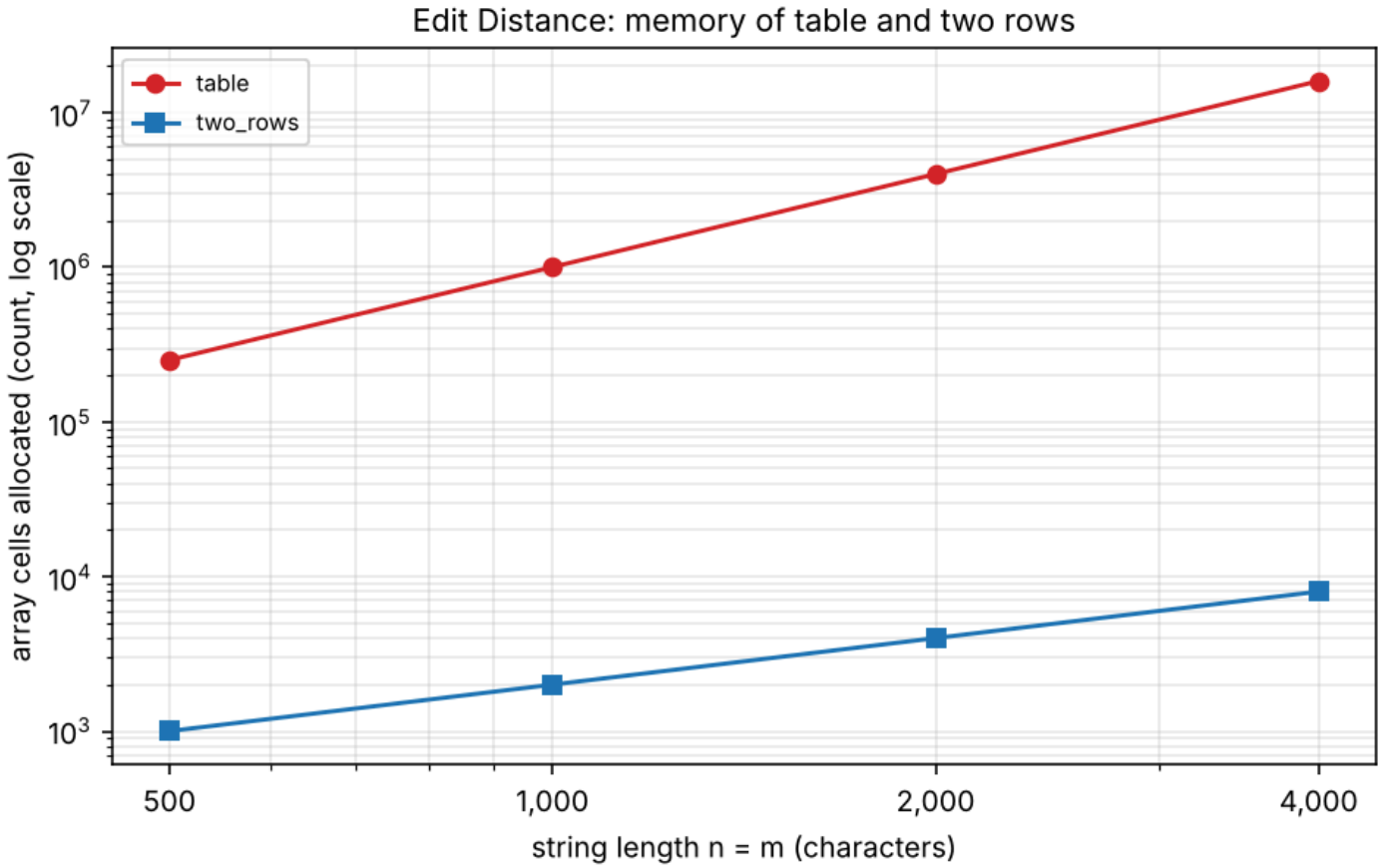


Figure 3. At n = 4000 the table needs 16 008 001 cells and two rows only 8 002, about 2000× less, and the gap grows with n.

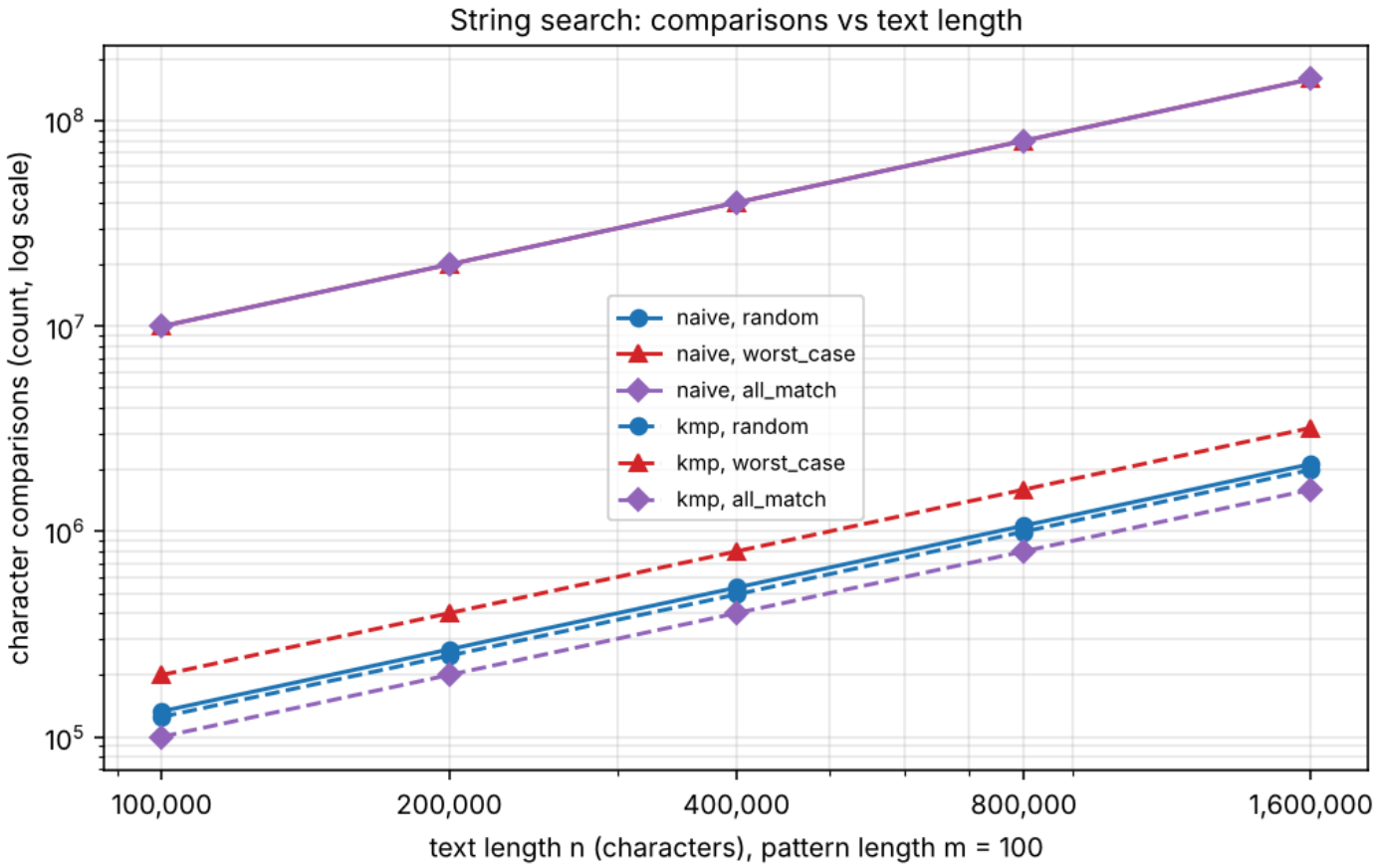


Figure 4. Naive does about 100·n comparisons on worst_case and all_match , KMP only 1–2·n; on random text both are close to 1.3·n

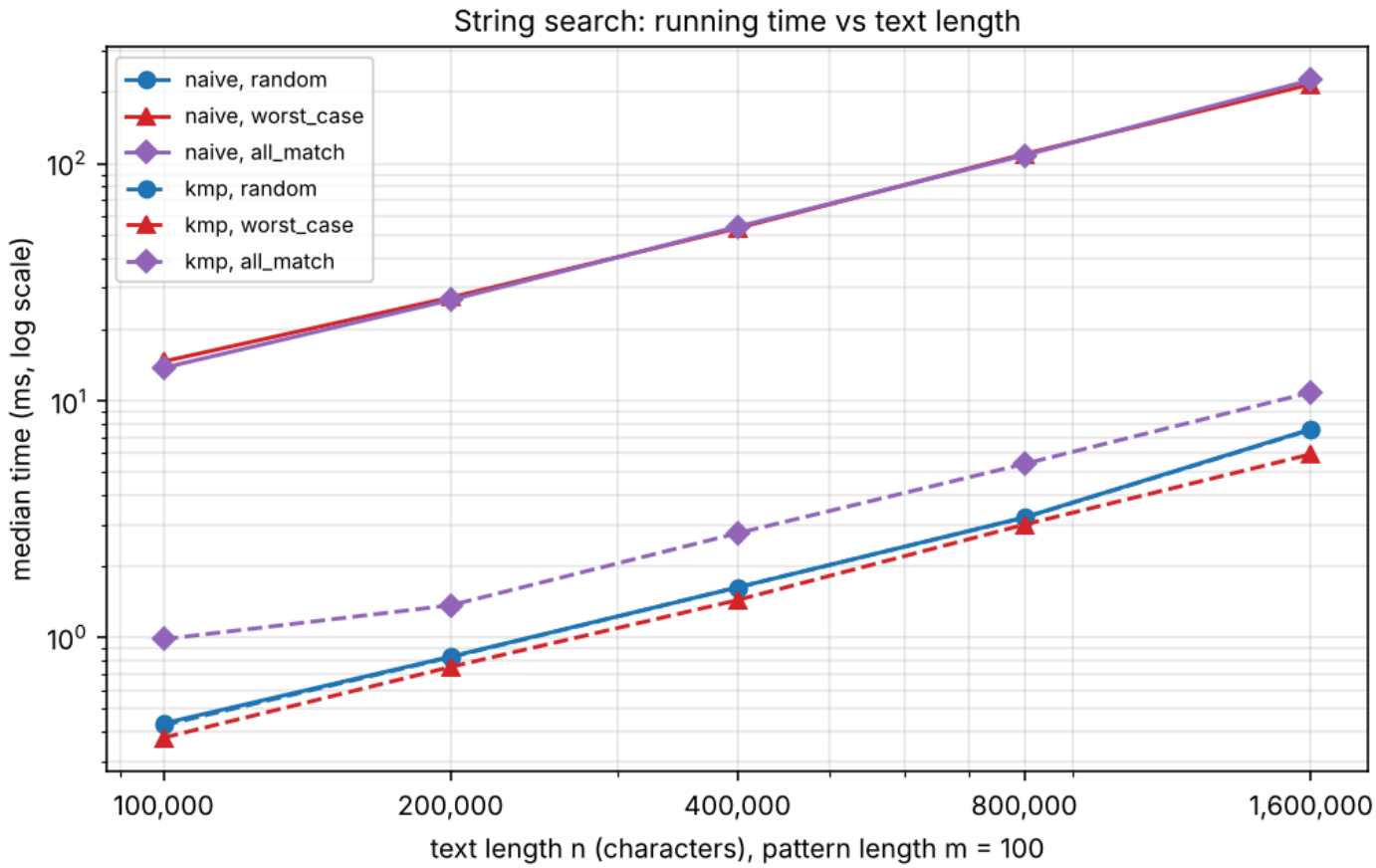


Figure 5. The time follows the comparisons: at n = 1.6 million Naive is 36× slower than KMP on worst_case (216 against 5.9 ms) and 21× on all_match , but equal on random text (7.6 against 7.5 ms).

3. Prediction vs Result

Edit Distance. Expected: recursive calls ×5.8 per character on disjoint , n+1 on identical ; table and two rows exactly $(n+1)^2$ cells on every input; time ×4 when n doubles; memory ratio ≈ n/2; two rows as fast or a bit faster.

Saw: the cell counts are exactly $(n+1)^2$ for both versions on all inputs (169 at n = 12, 16 008 001 at n = 4000) and the memory is 16 008 001 cells for the table against 8 002 for two rows at n = 4000, a ratio of 2000× – as predicted. The recursion on identical needed exactly n+1 calls. The growth on disjoint was ×172 for +3 characters, i.e. ×5.6 per character, a bit below 5.8, and random grew only ×3.6 per character (8.7 million calls at n = 12). I think 5.8 is the limit for large n and the small sizes are still below it, but I did not check larger n.

The time of two rows on random grew ×4.09, ×4.03 and ×4.04 for the three doublings of n – exactly as predicted. The table was less regular (×5.9, ×3.0, ×4.1); I do not know the reason, perhaps the allocation of the large table, but I did not check. Two rows were faster than the table in all 12 pairs of runs (by ×1.06 to ×1.52; at n = 4000 51.9 against 58.6 ms on random), which fits "as fast or slightly faster". At n = 4000 the times on the three input types are close (49–59 ms), so the input type did not matter for table and two rows, as the equal cell counts suggest.

String search. Expected: random – Naive ≈ 1.33·n, KMP ≈ n–1.3·n, equal time; worst_case and all_match – Naive ≈ 100·n, KMP ≈ 2·n. Saw: random : 1.33·n (Naive) and 1.25·n (KMP), times 7.58 and 7.54 ms at n = 1.6 million – as predicted. worst_case : 99.99·n against 2.00·n, a ratio of 50, and a time ratio of 36 (216 against 5.9 ms). Surprise 1: on all_match KMP needed only 1.00·n comparisons (every text character is compared once and there is no mismatch), not 2·n, so the comparison ratio is 100 and not 50. Surprise 2: KMP on all_match needs half the comparisons of worst_case (1.6 against 3.2 million) but is almost twice as slow (10.8 against 5.9 ms). Most likely the cost of building a list of 1.6 million boxed results dominates there; I did not measure it separately.

4. Short Answers

Which input made your string search slowest, and why?

For Naive it was all_match and worst_case : about 100 comparisons per text position (160 million comparisons at n = 1.6 million, 216–225 ms) against 1.33 per position on random . In both inputs the text is A…A and the pattern agrees with it for 99 or 100 characters, so Naive compares almost the whole pattern at every start position and then forgets everything and starts again

one position later. For KMP the most comparisons (2·n) came from worst_case , because after each mismatch it falls back inside the pattern and compares the same text character a second time; its slowest time was all_match (10.8 ms), which is probably explained by the large result list

How large an n could the recursive Edit Distance handle, and what stopped it?

On disjoint strings n = 12 took 0.83 s (377 million calls, about 450 million calls per second). I did not run larger sizes in the experiment, because the number of calls grows ×5.6 per extra character: n = 13 needs about 2.1 billion calls and n = 14 about 11.9 billion (counted by running the same code once). At my speed that is roughly 5 s and 26 s, n = 16 would take about 14 minutes and n = 18 about 7 hours (extrapolation, not measured). What stops it is time, not memory: the recursion depth is only 2n (24 at n = 12), but the same pairs (i, j) are solved again and again. On random strings it is much more benevolent (20 ms at n = 12), because equal characters shorten the paths.

What do you gain and lose with two rows instead of the full table?

I gain memory: 2(m+1) cells instead of (n+1)(m+1), which is 8 002 against 16 008 001 at n = 4000, and in my runs also some time (faster in all 12 pairs, for example 51.9 against 58.6 ms on random at n = 4000; probably less memory to allocate and touch, not verified). I lose the old rows, so I get only the number: I cannot walk back through the table to list which insertions, deletions and replacements were used. For a search box that only needs the distance this loss does not matter

Which string search would I choose for the city platform, and why?

KMP. Its cost is at most about 2·n comparisons whatever the input, so one repetitive log file cannot make a search 36 times slower, as it did for Naive on worst_case . On ordinary random text it was as fast as Naive (7.5 ms against 7.6 ms), so I pay nothing for the safety, and the extra memory is only the m cells of the LPS array. Naive would be acceptable only for short texts where the input is known to be random.

5. One Paragraph on Time

The measured times range from 0.38 ms (KMP, worst_case , n = 100 000) to 225 ms (Naive, all_match , n = 1.6 million) for string search, 49–59 ms for table and two rows at n = 4000, and 0.83 s for the recursion at n = 12 – all medians of 5 runs (3 for the recursion) after warm-up on one machine. They are not my only evidence because they depend on the laptop, the JIT, the garbage collector and the cache: the table and two rows evaluate exactly the same 16 008 001 cells, but one took 58.6 ms and the other 51.9 ms, and KMP on all_match is slower than on

worst_case although it makes half the comparisons. The counters do not have this problem: they are measured inside the algorithms and are identical on every run and machine, and they show the growth laws directly – $(n+1)^2$ cells, ×5.6 per character for the recursion, $100 \cdot n$ against $2 \cdot n$ comparisons. So the counters give the explanation and the times only confirm that the explanation matters in practice.